13 紅黑樹

Part III Data Structures

本章定位

紅黑樹用黑高與局部色限制保證對數高度；本章由NIL與精確父子接線，逐案證明插入三類、刪除四類及其鏡射，以相同鍵值前後圖核對額外黑、根分支與record身分。

前置知識

二元搜尋樹、旋轉與不變量證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以紅黑性質界定近似平衡 BST。
- 推導旋轉前後的中序不變性。
- 追蹤插入與刪除修正並驗證黑高。
- 13.1 Properties of red-black trees
- 13.2 Rotations
- 13.3 Insertion
- 13.4 Deletion

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先認識圖中的角色與路徑計數

深色節點是黑 B，淡紅節點是紅 R；鍵值後的 B／R 同時標出顏色。橘框標記目前修正位置，不代表第三種節點色。旋轉圖只談連結，不指定節點顏色。

情境	圖中的角色
旋轉	x：舊局部根；y：右 child；p：x 的舊父；α、β、γ：保留中序區間的子樹
插入	z：目前紅節點；p：父；g：祖父；u：叔
刪除	x：缺黑位置；p：父；w：兄弟；near：靠 x 的姪；far：遠離 x 的姪

x +1 只在證明中多算一黑。圖底的完整路徑黑數**含起點與末端 NIL**；$bh(x)$ 則不含 x。未畫的空 child 仍是黑 NIL；near／far 若為 NIL，會明畫。

13.1 Properties of red-black trees

BST 之外多一個 color，保證的是高度

每個內部節點存key、data、left、right、parent、color。以下示範互異key；重複插入回傳既有record。若要保留重複key，可改以(key,唯一ID)作嚴格總序，以下BST論證不變。

紅黑性質為：(1)每節點紅或黑；(2)根黑；(3)每個外部NIL葉黑；(4)紅節點的兩child皆黑；(5)任一點到其後代NIL的所有路徑黑節點數相同。最後一項是局部每個子樹都要成立，不只是整樹某兩條路相等。

用單一黑sentinel NIL表示全部空child及根的parent；空樹root=NIL。NIL沒有有效key/data，NIL.left=NIL.right=NIL便於安全讀color。正常節點非NIL的child必指回正確parent。所有NIL參照共用物件，不能同時宣稱每個空位置都有自己獨立的NIL.parent。

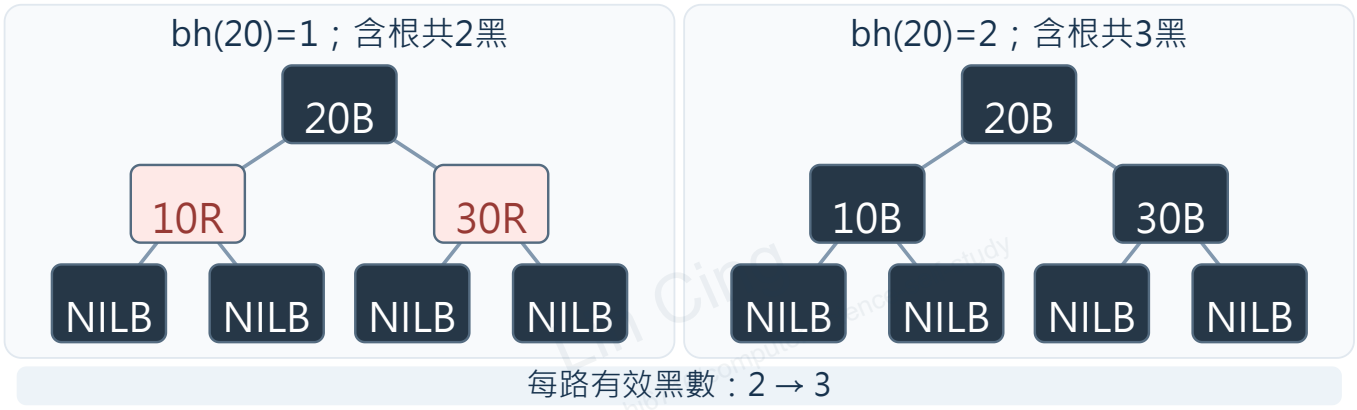
黑高定義：不含自己，但含末端 NIL

$bh(x)$為從x到任一後代NIL的黑點數，**不含x、含末端NIL**；$bh(NIL) = 0$。若child紅，$bh(child) = bh(x)$；若child黑，$bh(child) = bh(x) - 1$。這是依child是否被計入父的路徑決定，不可顛倒。

根20B、左右child10R及30R、其餘全NIL，則bh(20)=1、bh(10)=bh(30)=1，NIL為0。將兩child都改黑，則bh(20)=2，而bh(10)=bh(30)=1。每條路含根的黑數由2變3，但bh仍各排除其起點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

黑高定義：不含自己，但含末端 NIL （續）



本章h計root到NIL的最長邊數，空樹h=0；只有一個黑根時h=1。若採只算內部節點邊數的高度，非空時少1，不應在同一式中交換定義。

最小節點數與高度界的完整歸納

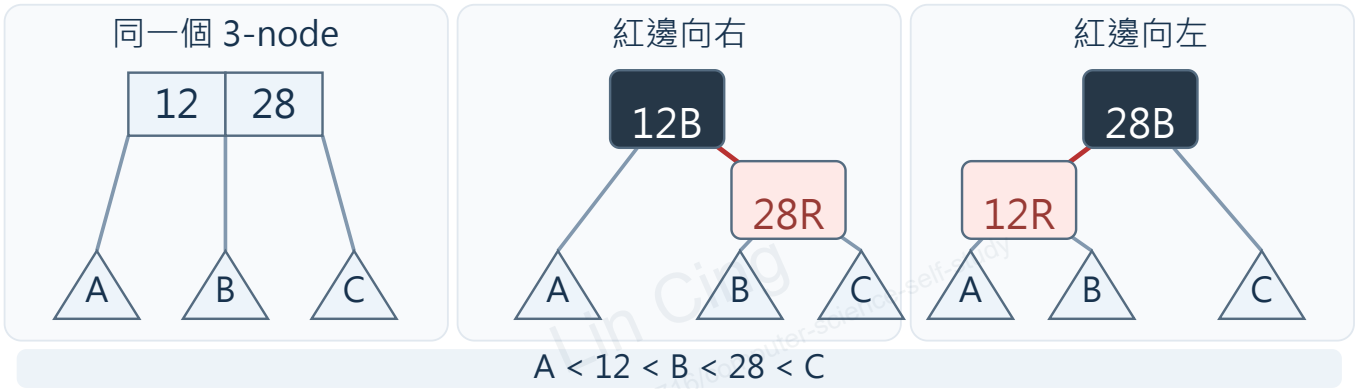
引理：以x為根的子樹至少$2^{bh(x)}-1$個內部節點。對高度歸納：x=NIL時0=2⁰−1；內部x的兩child高度較小，且黑高至少bh(x)−1，因此各至少$2^{bh(x)-1}-1$個內部節點（bh(x)≥1）。加上x得到

$$n_x \geq 2(2^{bh(x)-1}-1)+1 = 2^{bh(x)}-1.$$

取一條最長root-NIL路，含B個黑點與R個紅點。根與末端NIL黑，無相鄰紅；每個紅點可配給前面不同的黑點，故R≤B−1。路有h+1點，h+1=B+R≤2B−1，得h≤2(B−1)=2bh(root)。與節點引理合併，$n \geq 2^{h/2}-1$，故$h \leq 2\log_2(n+1)$。空樹亦滿足0≤0。

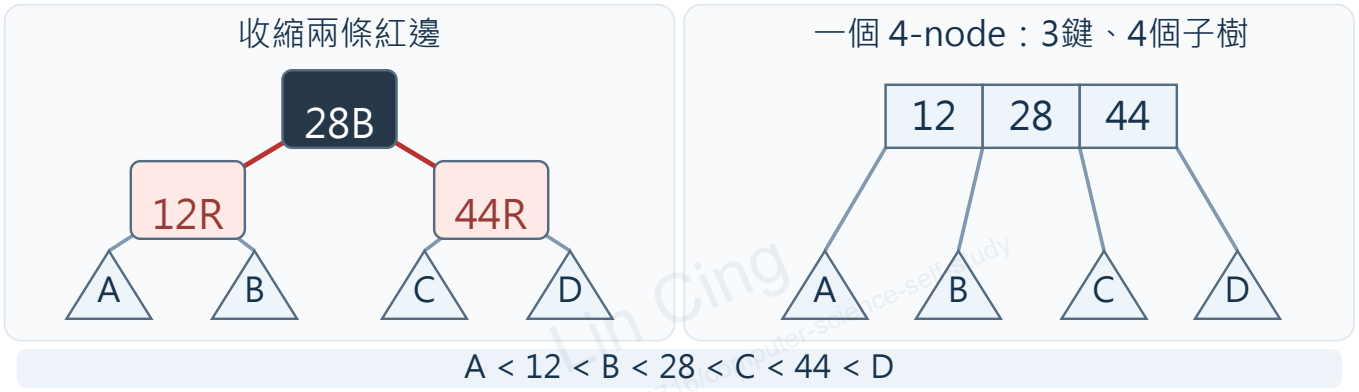
因此search/min/max/successor/predecessor走一條路需O(log(n+1))；插入、刪除還須證修正成本。輸出所有n點的中序、前序、後序走訪仍Θ(n)。紅黑樹也不要求左右高度差≤1，不能當作AVL的定義。

同一個 3-node，為何有兩種紅黑表示？



把紅 child（子節點）與黑 parent（父節點）合併，即為一個多鍵節點；單獨黑點則為 2-node。A、B、C 是黑根子樹（可為 NIL），黑高相同；不等式適用於各子樹內所有鍵。兩種表示中序相同，**一般紅黑樹不要求紅邊左傾**。

收縮紅邊後，黑高相等對應多元樹的等深葉



不允許連續紅邊，所以一個黑點至多與兩個紅 child 合併，得到 2／3／4-node；各子樹區間不變。每條 root–NIL 路扣除末端 NIL 一黑後，黑點數等於收縮後的多鍵節點數；各路黑數相等，外部葉便等深。紅邊採用 child 的色；根另外約定為黑。

13.2 Rotations

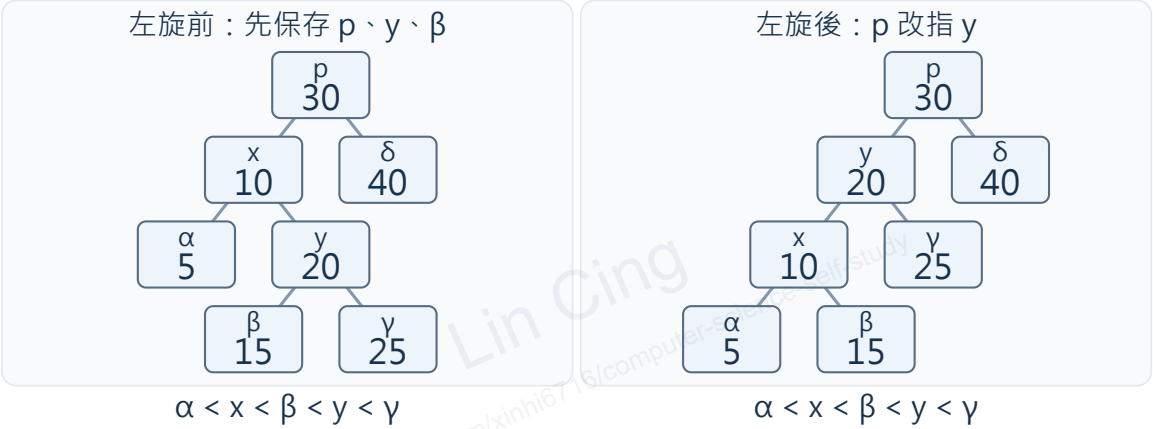
左旋先保存舊父，才接新局部根

前提x≠NIL、y=x.right≠NIL；令p=x.parent、β=y.left。α=x.left、γ=y.right的內部連結不動。

```
LEFT-ROTATE(T,x):
    p=x.parent; y=x.right; beta=y.left
    x.right=beta
    若 beta!=NIL: beta.parent=x
    y.parent=p
    若 p==NIL: T.root=y
    否則若 x==p.left: p.left=y
    否則: p.right=y
    y.left=x
    x.parent=y
```

必須更新**x的舊父p指向y**；y的舊父就是x，不可把這兩者混淆。β=NIL時不寫其parent，避免覆蓋刪除修正正在使用的共享sentinel上下文。旋轉本身不變更任何color、key或data。

13.2 Rotations （續）



指標表與右旋完整鏡射

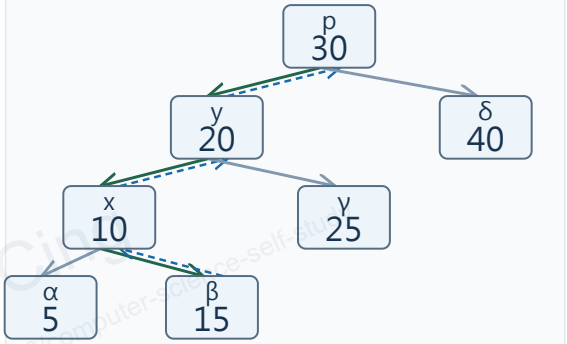
例p30、x10、y20、β15，x是p.left。順序寫10.right=15、15.parent=10、20.parent=30、30.left=20、20.left=10、10.parent=20。若p=NIL，則取代30.left的寫入是root=20，並令20.parent=NIL。

```
RIGHT-ROTATE(T,y):
    p=y.parent; x=y.left; beta=x.right
    y.left=beta
    若 beta!=NIL: beta.parent=y
    x.parent=p
    若 p==NIL: T.root=x
    否則若 y==p.right: p.right=x
    否則: p.left=x
    x.right=y
    y.parent=x
```

指標表與右旋完整鏡射 （續）

1. 10.right = 15
2. 15.parent = 10
3. 20.parent = 30
4. 30.left = 20
5. 20.left = 10
6. 10.parent = 20

實線 ↓ : child
虛線 ↑ : parent



若 p=NIL：步驟4改 root=20；步驟3為20.parent=NIL。

為何旋轉保序，為何不能單憑旋轉宣稱紅黑合法

左旋前後中序皆α,x,β,y,γ：α的鍵<x，β在x與y之間，γ>y；β改為x.right仍合法。根外只把p原指x的同一側改成y，其餘範圍不動。非NIL的β.parent、x.parent、y.parent分別改x、y、p，所以內外結構一致。右旋完全對偶。

只寫常數個欄位，時間O(1)。搜尋修復位置仍需O(log n)，不因單次旋轉常數就變O(1)插刪。旋轉保BST與record身分，**不自動保黑高或紅紅規則**；下一節的重上色必須配合局部黑數證明。

13.3 Insertion

紅葉插入只可能破壞根黑或父子紅紅

以普通BST搜尋空位置並建立z，z.left=z.right=NIL、z.color=R，再接parent。紅葉取代原黑NIL時，原路末端仍有同一個黑NIL，故不改所有相關黑數；若先插黑色，只有經此位置的路多一黑，會破壞性質5。

若空樹插第一點，它紅且為root，只需最後塗黑；否則僅可能z與紅父p相鄰。修正迴圈不變量為：z紅；若z.parent是root，該parent必黑；最多只有根紅或z/parent紅紅其中一種違規；其餘色性質與BST序均成立。進入迴圈時p紅，所以p不可能是根，祖父g存在，且g黑（否則另有一組紅紅）。

插入修正完整流程

以下p是g.left；另一半把left/right及LEFT/RIGHT-ROTATE全部交換。

```
當 z.parent.color==R:
    若 z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        若 u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        否則:
            若 z==z.parent.right:              // I2，接著做I3
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                   // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    否則: 執行完全左右鏡射的同三分支
T.root.color=B
```

插入修正完整流程 （續）

I2與I3不是互斥if；I2轉換後必落入I3。同時要用更新後的z.parent與祖父，不能再拿旋轉前的p當修復對象。

p是g.right時	右側的明確動作
I1：叔g.left紅	p、叔黑；g紅；z=g
I2：叔黑且z=p.left	z=p；RIGHT-ROTATE(z)，接I3
I3：叔黑且外側	新z.parent黑、祖父紅；LEFT-ROTATE(祖父)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I1：叔紅重上色的黑數不變

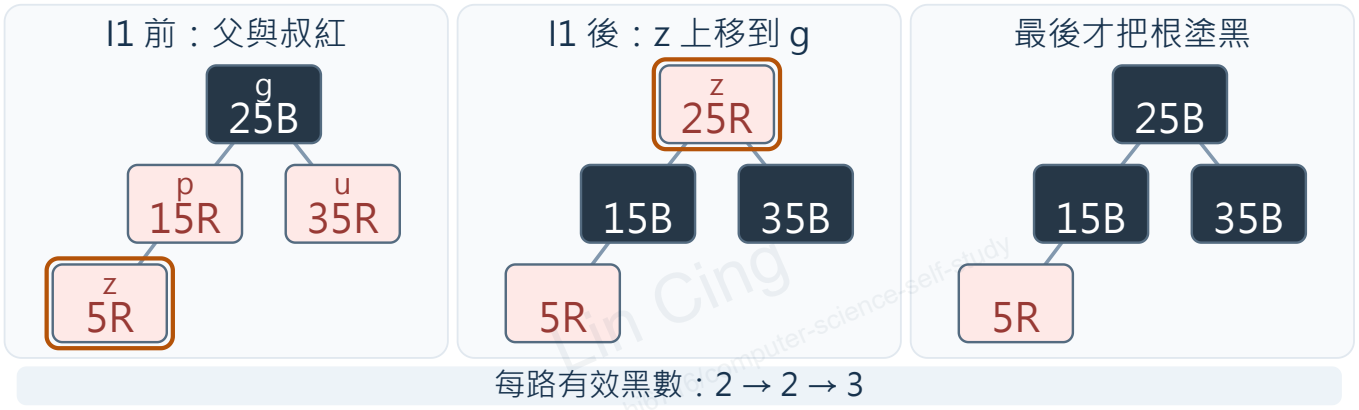
令父p與叔u以下每條路各提供b個黑點，包含末端NIL。原g黑、p/u紅，經g的每條完整路黑數為1+b；改g紅、p/u黑後仍1+b。雖然不含g的bh(g)由b變b+1，**祖先穿過g的完整黑數沒有增加**。

新z設g，紅紅只可能上移至g及其父；原z-p已因p黑消除。若g變成根，暫時只根紅，最後根黑才使全樹每條路同增一黑，不破相等性。

原創例依序插25、15、35、5：新5R的父15R、叔35R、祖25B。I1把15/35黑、25紅，z移25；parent為NIL黑，結束後root25黑。非根g時不得套用這最後同增一黑的根分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I1：叔紅重上色的黑數不變 （續）



每路有效黑數：2 → 2 → 3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I2：把內側折線轉成外側，不跳過下一步

叔黑且p=g.left、z=p.right時，先令z=p，再左旋舊p。原p與新節點皆紅，旋轉未改任何路上的黑點貢獻；BST中序保持。旋轉後舊p變成新局部根的左紅child，形成外側鏈，仍須I3才能消除紅紅。

插35、15、25：插25後root35B、左15R、15.right=25R。左旋15後root仍35B，35.left=25R，25.left=15R，算法z現在指15。紅紅由15/25保留，不能此時返回。



每路有效黑數：2 → 2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

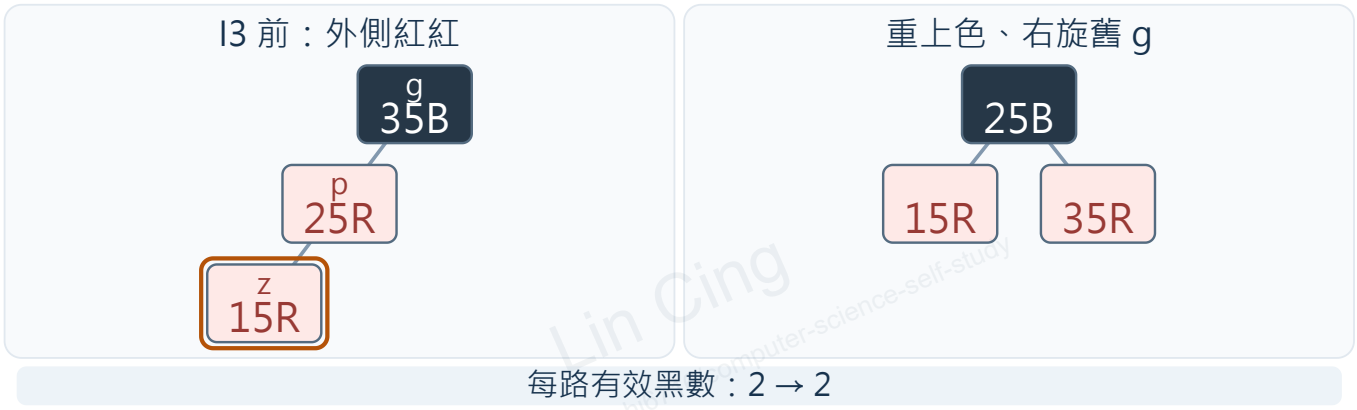
I3：重上色加旋轉，恢復全部局部路徑

外側鏈gB-left pR-left zR，叔黑。將p黑、g紅，右旋g，使p成局部根。設α、β為z兩child，γ為p原right，δ為g原right；四棵邊界子樹均黑根（可為NIL），各完整向下路提供b黑點。原經g的完整路為1+b；改後經新p亦1+b。z仍紅，但父p黑；g紅的兩childγ、δ皆黑，故沒有紅紅，局部根p黑也不會與外父衝突。

續前例：25塗黑、35塗紅，再右旋35，得到25B、左15R、右35R。中序15,25,35保留，bh(root)=1，每條root-NIL路含2黑。鏡射例插15、35、25，先右旋35，再左旋15，同得這棵樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I3：重上色加旋轉，恢復全部局部路徑 （續）



I1每次z上移兩層且不旋轉；I2+I3做至多兩次旋轉後必結束。故重上色O(log(n+1))、旋轉最多2次，連BST插入總O(log(n+1))。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.4 Deletion

先分清 z、y、x 與 y 的原色

z是要刪除的record；若z少於兩child，y=z；若有兩child，y是z.right中的最小點，會整個移到z位置。x是填補**y原位置**的唯一可能child（若y為後繼，取y.right，因後繼沒有left；若y=z，取z唯一非NIL的child，兩child皆NIL時取NIL）。要保存yOriginalColor，再把移來的y.color設z.color；是否缺黑取決於yOriginalColor，不是移動後y的色，也不是只看z.color。

```
RB-TRANSPLANT(u,v):
    若 u.parent==NIL: root=v
    否則若 u==u.parent.left: u.parent.left=v
    否則: u.parent.right=v
    v.parent=u.parent              // 即使 v==NIL 也寫
```

這次NIL.parent保存當前x空位置的parent，供找sibling使用；它不是全部空child的通用父。根parent=NIL。旋轉遇到β=NIL則不寫β.parent，以免覆寫這個x上下文。穩定操作結束後可重設NIL.parent=NIL；color必始終黑。

RB-DELETE 的完整物件移接

```
RB-DELETE(z):                  // z 是已找到的有效record
    y=z; yOriginalColor=y.color
    若 z.left==NIL:
        x=z.right; RB-TRANSPLANT(z,z.right)
    否則若 z.right==NIL:
        x=z.left; RB-TRANSPLANT(z,z.left)
    否則:
        y=MINIMUM(z.right); yOriginalColor=y.color; x=y.right
        若 y.parent==z:
            x.parent=y          // 包含x為NIL的分支
        否則:
            RB-TRANSPLANT(y,y.right)
            y.right=z.right; y.right.parent=y
        RB-TRANSPLANT(z,y)
        y.left=z.left; y.left.parent=y
        y.color=z.color
    若 yOriginalColor==B: RB-DELETE-FIXUP(x)
```

本章以整個y物件移動，y自己的key/data與handle保持，不把z的data錯套給y。API可先SEARCH(key)，找不到直接NOT_FOUND而不修改；上面程序的前提是z存在，不對NIL作普通刪除。

後繼移接的具體位置與 NIL parent

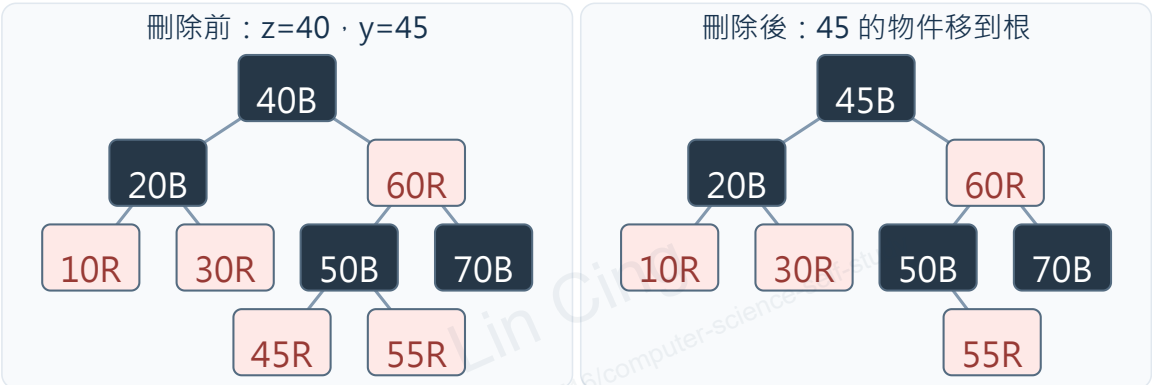
依序插40、20、60、10、30、50、70、45、55，得到root40B，左20B(10R,30R)，右60R，其左50B(45R,55R)、右70B。刪z40時y45R，x=NIL，y.parent50≠z。

先移除50.left的45，令50.left=NIL且NIL.parent=50；y.right接60並改60.parent=45；用y替root40，再接y.left=20、20.parent=45，令y.color=B。Y原色R，所以不修黑；root45B，原45的data仍跟隨45，只有40的record被移除。

若y就是z.right，則不可先把x.parent留在將刪的z，必指定x.parent=y。這兩個分支對非NIL與NIL的x都要成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後繼移接的具體位置與 NIL parent （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

缺一黑以 x 位置追蹤，不新增第三種 color

若y原紅，移除不減黑數，y在新位置取z原色，其他局部紅黑關係亦保留；無須fix。若y原黑，把它的一份黑暫時計到x位置，令所有經x的路多算一黑，其他路的黑數才與刪前一致。

這是證明用的「extra black」，實際x.color仍只有R/B。X紅時直接塗黑吸收extra；x是root時可把extra從全樹路徑一起去掉；其餘迴圈x為黑且非根。兄弟w不能NIL，否則它只有一個普通NIL黑，而x分支至少有NIL加extra兩黑，與有效黑數相等矛盾。

以下x在p.left，w=p.right；near=w.left、far=w.right。右側缺黑時全部左右交換：near=w.right、far=w.left，旋轉方向也反轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DELETE-FIXUP 的完整左側流程

```
當 x!=root 且 x.color==B:
    若 x==x.parent.left:
        w=x.parent.right
        若 w.color==R:                        // D1
            w.color=B; x.parent.color=R
            LEFT-ROTATE(T,x.parent)
            w=x.parent.right
        若 w.left.color==B 且 w.right.color==B: // D2
            w.color=R
            x=x.parent
        否則:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DELETE-FIXUP 的完整左側流程 （續）

```
            若 w.right.color==B:               // D3，接著做D4
                w.left.color=B; w.color=R
                RIGHT-ROTATE(T,w)
                w=x.parent.right
            w.color=x.parent.color             // D4
            x.parent.color=B; w.right.color=B
            LEFT-ROTATE(T,x.parent)
            x=root
    否則：執行完全左右鏡射的同四分支
x.color=B
```

D1之後重新取w，D3之後也重新取w；分類不是四個互斥elif。單一迭代可能D1→D2，或D1→D3→D4；後兩者與直接D4都會終止。

DELETE-FIXUP 的完整左側流程 （續）

x是p.right時	w=p.left，near=w.right，far=w.left
D1：w紅	w黑、p紅；RIGHT-ROTATE(p)；重取w
D2：w兩child黑	w紅；x=p，上移或最後塗黑吸收
D3：far黑、near紅	w.right黑、w紅；LEFT-ROTATE(w)；重取w
D4：far紅	w取p原色；p與w.left黑；RIGHT-ROTATE(p)；x=root

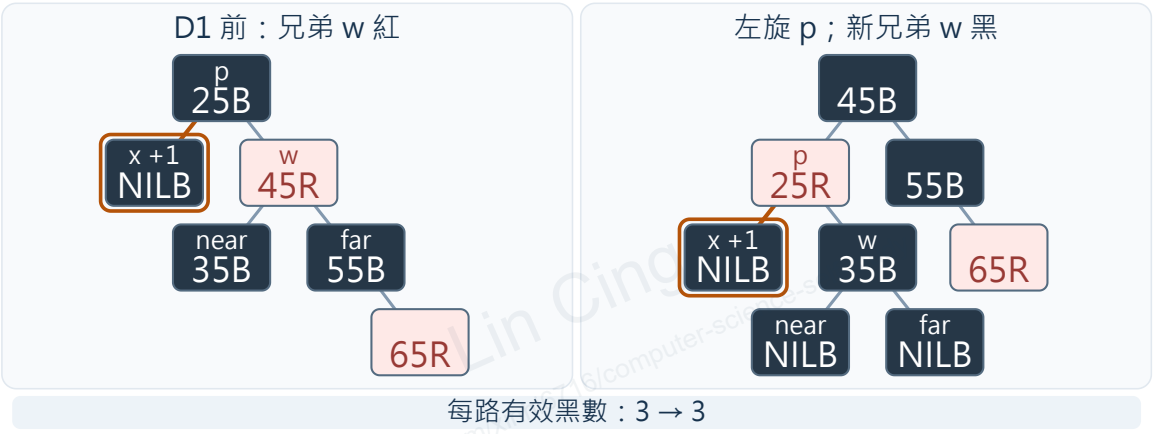
D1：紅兄弟轉成黑兄弟，extra 仍在原位置

共同計數記x分支**實際**完整黑數b，含x（若NIL便含NIL）；加extra後b+1。D1的w紅迫使p及w兩child黑。w每個child的完整黑數為b+1，所以旋轉前從p含自己的有效黑數皆b+2。

令w黑、p紅、左旋p。新局部根w黑，其左p紅下的x有效b+1與w原近child b+1相等，右側w原遠child亦b+1，所以新局部總仍b+2。Extra沒有被消除，x的新兄弟為原w的黑近child；因此只是轉成D2／D3／D4。

數例依序插15、25、35、45、55、65，再刪15。移除後x=NIL是25B.left，w45R；D1將45黑、25紅，左旋25。新root45B，左25R(right35B)、右55B(right65R)，x仍為25.left。根或祖先的完整黑數不因這個轉換改變。

D1：紅兄弟轉成黑兄弟，extra 仍在原位置 （續）



D2：兩個黑 child，缺黑向上或被紅父吸收

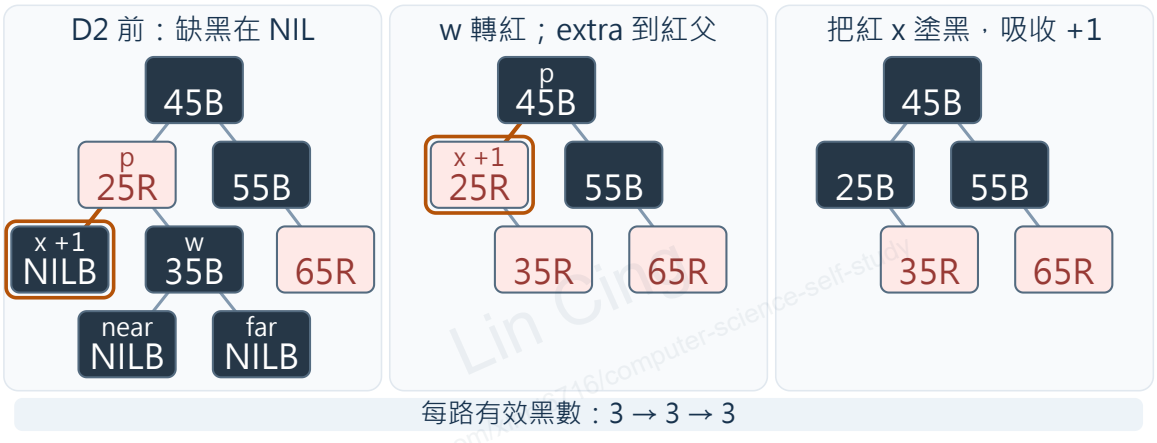
此時w黑且near/far都黑。令x實際黑數b、有效b+1，w完整黑數b+1、每個child完整黑數b。W改紅後其完整黑數變b，兩邊**實際**黑數恢復相等，但整個p子樹相對祖先少一黑，故將extra改附p，即x=p。

若p原紅，最後p塗黑恰補一黑，結束；若p原黑且非root，再迭代上移；若p是root，從所有路一起去extra，結束。D1剛把p變紅，所以D1後若接D2必會在這一輪後吸收，不會再上升到另一次D1。

續前例：w35B兩child NIL，改35R，x由NIL改25R，退出迴圈後25黑。最後45B的左25B(right35R)，右55B(right65R)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D2：兩個黑 child，缺黑向上或被紅父吸收 （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D2 的根分支：整棵樹可同降一黑

若原樹為25B、兩黑葉15B/35B，刪15則D2令35紅、x移root25，從全部根葉路一起去掉extra，最後25B(right35R)。Root的bh從2降1，所有路仍相等；這與紅父塗黑吸收是不同的結束分支。



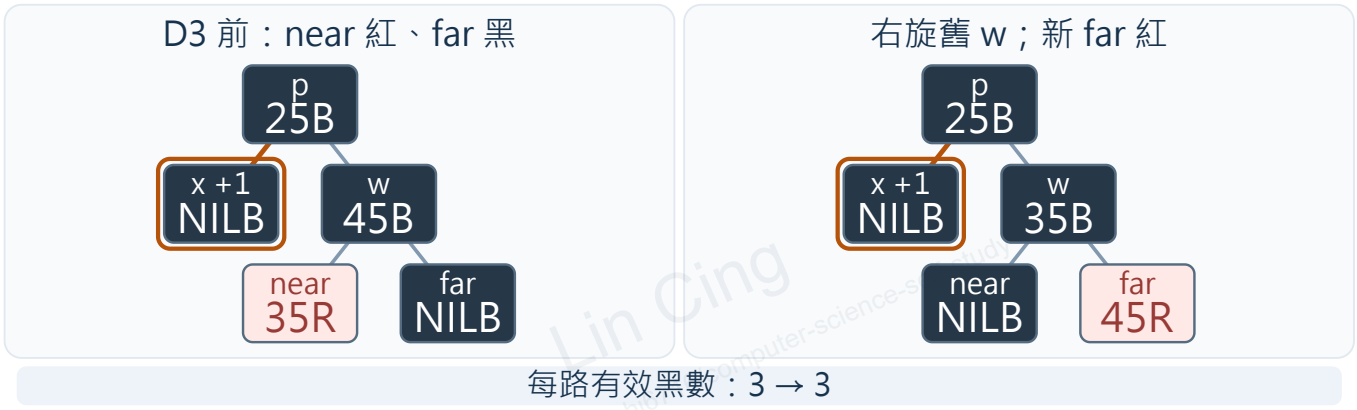
D3：近紅遠黑，先把可用紅色轉到遠端

W黑、near=N紅、far黑。令x實際黑數b；w兩child的完整黑數皆b。N紅的兩child各完整黑數b，far亦b。將N黑、w紅並右旋w：新兄弟N黑，其left舊N.left為b，right舊w紅的兩child舊N.right及far亦b，所以新兄弟完整黑數仍b+1；p以上不變。

新兄弟的far就是舊w，顏色紅，故立即進D4，extra仍在x。此步沒有「上移缺黑」，也未終止。

數例插15、25、45、35後root25B、左15B、右45B(left35R)。刪15後x=NIL，w45B、near35R、far=NIL黑。35黑、45紅，右旋45，得到root25B、右35B(right45R)，接著必D4。

D3：近紅遠黑，先把可用紅色轉到遠端 (續)



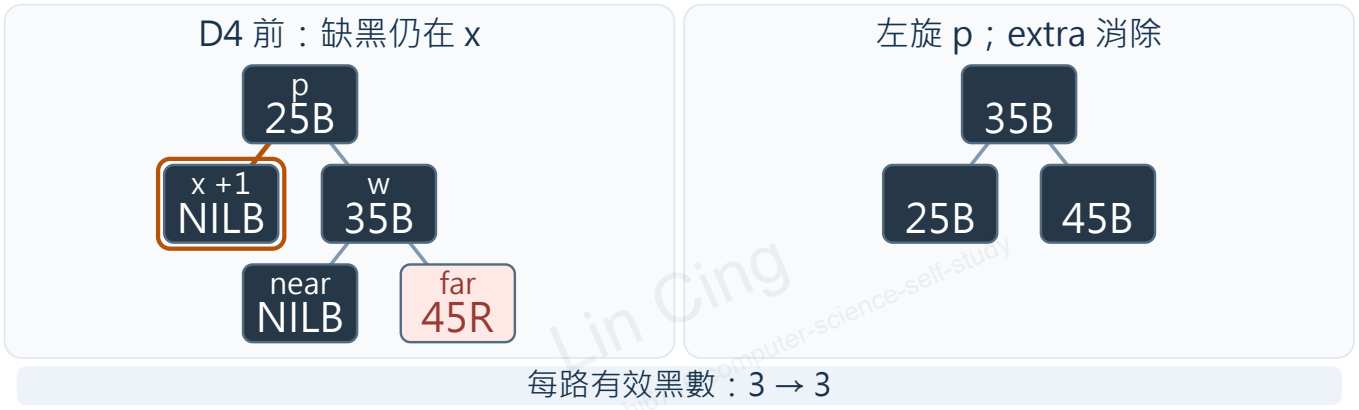
D4：遠紅，旋轉後兩側直接補齊

令p原色的黑貢獻c∈{0,1}，x實際黑數b，w黑的near與far完整黑數皆b；far原紅。令w.color=p原色、p黑、far黑，左旋p。新根w左子p黑，p下的x（已不再加extra）與near皆b，所以左子樹完整黑數b+1；右far由紅轉黑，也為b+1。新局部總c+b+1，等於原p下含extra的有效總c+(b+1)。

若新w紅，其兩child p及far皆黑；原p變黑不會新造紅紅，w對外保留p的原色，所以外父關係亦保留。Extra已被結構與色變消除，令x=root結束，再保證root黑。

續D3數例：w35繼承25原黑，25黑、far45黑，左旋25，得到35B、兩child25B及45B。三條key為25,35,45，每條root-NIL含3黑，不再有extra。

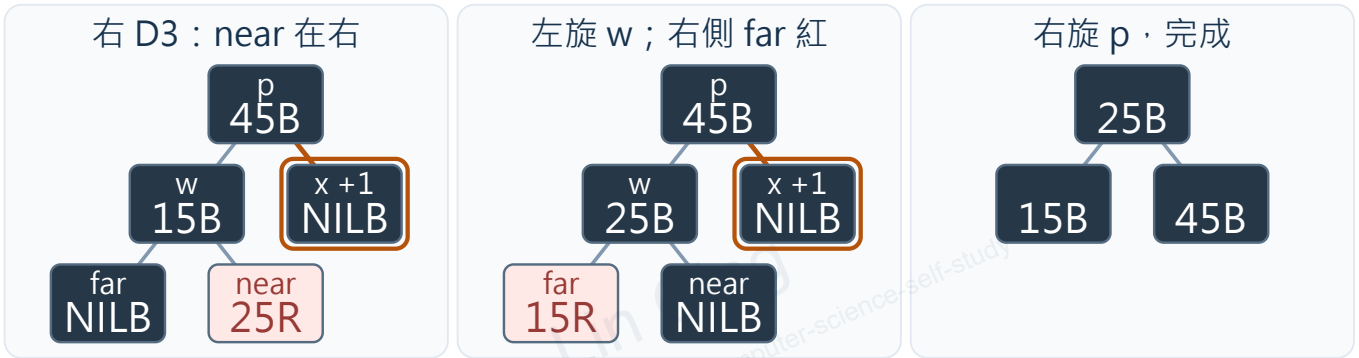
D4：遠紅，旋轉後兩側直接補齊 （續）



每路有效黑數：3 → 3

鏡射例插15、35、45、25後刪35：後繼45B直接接新根，x=NIL是45.right，所以NIL.parent必設45。兄弟15B的near25R、far=NIL；右側D3左旋15，右側D4右旋45，最後25B、兩child15B及45B。

D4：遠紅，旋轉後兩側直接補齊 （續）



每路有效黑數：3 → 3 → 3

同一個 D3→D4 例子，逐路把黑數加出來

以下每條路**含起點與 NIL**。D3 不改 x=NIL 的位置；D4 才消除 extra。

階段與路徑	實際黑數	extra	有效總數
D3 前：25B → NIL(x)	2	1	3
D3 前：25B → 45B → 35R → NIL	3	0	3
D3 後：25B → 35B → 45R → NIL	3	0	3
D4 後：35B → 25B → NIL	3	0	3
D4 後：35B → 45B → NIL	3	0	3

黑點各加 1、紅點加 0，只有穿過 x 的路再加 extra。未列出的另一側 NIL 路依同一規則也為 3；可對照前後圖逐一驗算。D4 後無須 extra，兩側已各補齊一黑。

四類守恆與總成本如何接成一般證明

BST移接先保住序與全部未刪record。若移除紅則直接合法；移除黑時，以x位置的extra讓祖先到葉黑數暫時平衡。每輪D1與D3保持同一缺黑位置及有效黑數；D2把唯一extra上移；D4完全消除。紅紅只可能在紅x吸收時，最後塗黑修復；w變紅的D2其兩child本黑，旋轉重上色也已逐案檢查。因此終止時五性質全部恢復。

D2每次沿祖先上移一層，最多O(log(n+1))次且不旋轉。D1之後p紅，若D2便吸收、否則經D3/D4結束；故D1最多一次，D3最多一次，D4最多一次，旋轉總最多3次。加搜尋z、搜尋後繼及移接，刪除總O(log(n+1))。NIL/root/紅child直接吸收為合法基例，刪最後一點得到空黑root=NIL。

這是任意合法紅黑樹的一般證明；有限程式另逐次檢查全域鍵序、雙向parent、紅child黑、所有NIL路黑數、root色及未刪record物件身分，並包含左右各插入3類、刪除4類。重上色可沿高度發生，不能把「最多3次旋轉」寫成整個刪除O(1)。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 13。

Lin Cing，個人資料結構手寫筆記，紅黑樹與2-3-4 tree對照；本章依其觀念重新推導、繪圖。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study